온디바이스AI 시험 대비 — 이론 정리

범위: 1. Linux & Python · 2. Computer Vision 기초 · 3. 딥러닝 기초 및 GPU 가속

1. Linux 기본 개념

- **CLI vs GUI**: GUI는 아이콘·마우스로, CLI는 명령어 입력으로 조작. Jetson 등 Linux 환경에서는 주로 CLI 사용.
- **터미널** = 명령어를 입력하는 창 / **셸(bash)** = 명령을 해석·실행하는 프로그램.
- **프롬프트** `user@host:~/parent/current$` — `@` 사용자·호스트 구분, `~` 홈 디렉토리, `/` 경로 구분, `$` 일반 사용자, `#` 관리자(root).
- **권한**: `sudo`는 관리자 권한으로 실행. `chmod` 숫자 표기는 r=4, w=2, x=1이고 소유자/그룹/기타 순 (예: 644 = 소유자 rw, 나머지 r).

2. 이미지의 구조

- 이미지는 **H×W×C NumPy 배열**이고 [row, col, channel]로 접근. 원점은 **좌측 상단** (row↑ → 아래, col↑ → 오른쪽).
- 기본 타입은 **uint8 (0~255)**. 연산할 때 float32로 바꾸면 **반드시 /255.** → 0~1 범위. 나누지 않으면 Matplotlib이 1 넘는 값을 잘라내서(clipping) 하얗게 보임.
- 배열이므로 NumPy **슬라이싱 = crop**.

3. 색공간 (Color Space)

- 색을 일정한 기준으로 표현하는 방식. 색공간마다 표현 범위가 다르고, 목적에 맞게 변환하면 전처리 효율이 올라감.

색공간	특징	용도·주의
RGB / BGR	같은 정보, 채널 순서만 다름. **OpenCV = BGR**, **Matplotlib = RGB**	시각화 도구 기준에 맞게 변환. 안 하면 빨강↔파랑이 뒤바뀜
Grayscale	Gray = 0.299R + 0.587G + 0.114B 사람 눈이 초록에 가장 민감 → G 가중치가 가장 큼	RGB 이미지를 BGR2GRAY로 변환하면 R·B 가중치가 바뀌어 밝기가 달라짐
HSV	H 색상, S 채도, V 명도가 **독립 분리** H는 각도라서 360°를 넘으면 0으로 **순환** OpenCV: uint8은 H 0~179, S·V 0~255 / float32는 H 0~360, S·V 0~1	특정 색 추출(Segmentation)에 유리. 빨강은 0° 근처와 180 근처 양쪽 → 마스크 2개를 OR로 합침. H가 0~179인 이유: 360은 8비트(최대 255)에 안 들어가서 절반으로 줄임
LAB	대립색 이론 기반. L* 지각 밝기(0~100), a* 초록↔빨강, b* 파랑↔노랑 밝기(L)와 색(a, b)이 **완전 분리**	**조명 오염에 강함**. CLAHE를 L 채널에만 적용

4. 필터링 (Filtering)

- 노이즈 같은 불필요한 성분을 제거하고 원하는 특징만 골라내는 전처리. 커널을 이미지 위로 이동시키며 **합성곱(Convolution)**을 계산. 커널 크기는 중심 픽셀이 있도록 **홀수**.
- **Smoothing (LPF)**
  - **Box Filter**: 커널 안의 값을 단순 평균 → 블록 같은 흐림
  - **Gaussian Filter**: 중심에 가까울수록 가중치가 큼 → 더 자연스러운 흐림

    - 커널이 클수록 더 많이 흐려짐
- **Sharpening**: 중심은 크게, 주변은 음수인 커널로 중심과 주변의 차이를 키움. 커널 합이 1이면 전체 밝기 유지. OpenCV에 전용 함수가 없어서 직접 만든 커널을 `filter2D`에 넣음.
- **Border Type**: 커널이 이미지 밖으로 나가는 테두리를 채우는 방식. CONSTANT(0으로 채움), REFLECT / REFLECT_101(거울 반사), REPLICATE(끝 픽셀 반복). **CONSTANT**만 테두리가 검게 물들어 결과가 확연히 다름.

5. 이미지 주파수와 Edge Detection

	저주파 (Low)	고주파 (High)
의미	밝기가 천천히 변하는 영역 (하늘, 피부, 벽, 기본 형태)	밝기가 급격히 변하는 영역 (경계, 무늬, 질감, 노이즈)
제거하면	윤곽선 스케치만 남음	흐릿한 이미지가 됨
필터	**LPF**: Box, Gaussian (블러)	**HPF**: Residual, Unsharp, Sobel, Canny (엣지)

- **Residual Extraction**: 원본 − 블러(저주파) = 고주파. 가장 단순한 HPF.
- **Unsharp Masking**: 원본 + 고주파 → 경계 강조. 이름은 "흐린(unsharp) 이미지를 마스크로 사용"한다는 데서 옴.
- **Sobel Filter**: 밝기의 **1차 미분**(변화율)으로 edge 검출.
    - **dx** (x방향 미분) → 좌우 밝기 차이 → **세로 edge** / **dy** (y방향 미분) → 위아래 밝기 차이 → **가로 edge**
    - **magnitude = $\sqrt{(dx^2 + dy^2)}$**: 대각선을 포함한 방향 무관 전체 edge 강도
    - 부호에 방향 정보가 있음 (양수 = 어두운 곳 → 밝은 곳)
    - 장점: 매우 빠름 / 단점: 노이즈에 약하고 edge가 두꺼움

Canny Edge Detection — Sobel의 단점(두꺼운 edge, 노이즈)을 보완

1. **Gaussian Smoothing**: 미분은 노이즈를 증폭하므로 먼저 노이즈 제거
2. **Sobel**: edge 후보와 gradient 방향 계산 (아직 굵음)
3. **NMS (비최대 억제)**: gradient 방향으로 국소 최대값만 남기고 나머지는 0 → edge가 얇아짐
4. **Hysteresis Thresholding**: 임계값 2개 사용. 상한보다 크면 확실한 edge, 하한보다 작으면 제거, 그 사이 값은 확실한 edge와 **연결되어 있을 때만** 남김

결과: 얇고 정확하며 끊김이 적은 edge. ※ 입력은 **uint8 (CV_8U)**만 가능 (float를 넣으면 에러)

6. Color Segmentation과 Morphology

- **Color Segmentation**: 원하는 색만 분리·추출. 방법은 색상 임계값 처리(Color Thresholding), 결과는 **이진 마스크**(흰색 = 1, 검정 = 0). H 하나로 색을 지정할 수 있어서 보통 HSV에서 처리.
- 마스크 생성 전에 **약한 블러** → 픽셀 단위 노이즈 감소. 너무 세게 걸면 정보 유실.

연산	원리	효과
Erosion (침식)	커널 안이 **모두** 1일 때만 중심을 1로 (AND)	흰 영역 감소, 배경의 작은 흰 노이즈 제거
Dilation (팽창)	커널 안에 **하나라도** 1이면 중심을 1로 (OR)	흰 영역 확장, 구멍·끊긴 선 연결
Opening (열기)	침식 → 팽창	배경 노이즈 제거, 본체 크기 유지
Closing (닫기)	팽창 → 침식	내부 구멍 메움, 본체 크기 유지

- 일반적인 순서: **Opening으로 노이즈 제거 → Closing으로 내부 채우기**. Opening을 너무 많이 반복하면 본체가 끊어짐.

7. Image Enhancement — CLAHE

- Color Segmentation은 **조명 오염**(반사, 그림자)에 취약 → 대비를 강화해서 원하는 영역을 더 잘 추출.

단계	방식	문제·해결
Histogram Equalization	이미지 전체의 밝기 분포를 고르게 재분배	전역 처리 → 일부 영역이 지나치게 밝거나 어두워짐
Adaptive HE	격자(tile)로 나눠 각각 평활화	국소 대비는 향상되지만 단색 영역의 노이즈가 극대화
Contrast Limited AHE	히스토그램 높이가 상한(clipLimit)을 넘으면 잘라서 재분배	노이즈 과증폭 방지, 과하지 않은 명암비

- 적용: **LAB의 L 채널에만** 적용 → 색(a, b)은 그대로 두고 밝기 대비만 강화. LAB에서 HSV로 바로 변환할 수 없어서 RGB를 거쳐 HSV로 변환.

8. Contour와 특징점

- **Contour (외곽선)**: 이진 마스크에서 연결된 영역의 경계선. 영역이 이어져 있어야 하나의 contour가 되므로 그 전에 morphology를 신중하게 적용해야 함.
- 노이즈는 보통 작은 영역을 만듦 → **면적이 가장 큰 contour**를 목표 객체로 선택.
- RETR_EXTERNAL = 바깥 테두리만 검출 / CHAIN_APPROX_SIMPLE = 꼭짓점만 저장.
- **최소 외접원**: contour의 모든 점을 포함하는 가장 작은 원 → 중심과 반지름.
- **무게중심 (Centroid)**: 픽셀 분포 기반의 기하학적 중심. **Cx = M10/M00, Cy = M01/M00** (M00 = 면적). 반지름은 구할 수 없어서 최소 외접원의 반지름과 함께 사용.
- **Interest Point (특징점)**: 주변과 구별되는, 정보량이 높은 지점(코너 등). 크기·회전·조명 변화에 비교적 안정적이라 매칭·추적에 활용. 대표 기법: Harris, SIFT, ORB.

Harris Corner Detection: Sobel로 Ix, Iy → Ix^2, Iy^2, IxIy에 Gaussian → Response 계산 → Thresholding → NMS → 코너
$R = det(M) - \alpha \cdot trace^2(M)$ | R이 크면 **코너**, 음수면 **edge**, 0 근처면 **평탄한 영역**. 코너는 두 방향 모두로 밝기 변화가 큰 지점.

9. 카메라와 추적

- **USB 카메라**: 이미 MJPG/YUYV로 인코딩된 영상을 출력 → v4l2src로 바로 받음.
- **CSI 카메라**: 센서의 원본 **Bayer 데이터**(RG10) 출력 → **ISP**(Image Signal Processor) 처리가 필요 → **nvarguscamerasrc** 사용.
- **GStreamer 파이프라인**: 처리 단계를 `!` 로 연결한 흐름. 마지막 appsink에서 OpenCV가 프레임을 받음. drop / max-buffers=1 / leaky 옵션으로 오래된 프레임을 버려 지연(latency) 감소.

Kalman Filter: 노이즈가 섞인 측정값으로부터 실제 상태(위치, 속도)를 추정
- **예측 (Prediction)**: 운동 모델(등속 가정)로 다음 상태와 불확실성 예측
- **보정 (Update)**: 측정값이 들어오면 칼만 이득 K로 예측과 측정을 가중 결합
- 객체가 가려지거나 측정에 실패하면 → **예측만으로** 추적 유지
- 프로세스 노이즈가 크면 측정 변화에 민감하게 반응 / 측정 노이즈가 크면 측정을 덜 믿고 예측을 더 따름
- OpenCV: 상태 [x, y, vx, vy], 측정 [x, y] → `KalmanFilter(4, 2)`

- **MediaPipe**: 손 랜드마크 21개 (손가락 끝 4, 8, 12, 16, 20). 좌표는 0~1로 정규화되어 있어서 w, h를 곱해야 픽셀 좌표. 규칙 기반 예: 관절 세 점의 각도가 160° 이상이면 손가락이 펴진 것으로 판단.

10. 인공신경망 기초

- **딥러닝**: 뇌의 신경망을 본뜬 인공신경망으로 복잡한 데이터를 학습. **ANN** = 입력층 – 은닉층 – 출력층, 최소 단위는 **퍼셉트론**.
- **퍼셉트론**: $z = w_1x_1 + w_2x_2 + b$, 출력 y = f(z). 가중치 = 각 입력의 중요도, 편향 = 얼마나 쉽게 활성화될지(1을 출력할지)의 기준.
- **단층 (SLP)**: 결정 경계가 직선 → **선형 분리 문제만** 해결 (AND, OR, NAND, NOR).
- **XOR**은 직선 하나로 나눌 수 없는 비선형 문제 → 은닉층을 추가한 **다층 퍼셉트론(MLP)**이 필요. **XOR = AND(NAND, OR)**
- 단일 뉴런으로 XOR 학습 → 확률 약 0.5, 정확도 50%. 모델 구조가 문제에 맞지 않으면 데이터나 epoch를 늘려도 학습되지 않음.

게이트	가중치	편향	출력 (00, 10, 01, 11)
AND	양수	절댓값이 큰 음수	0 0 0 1
OR	양수	AND보다 작은 음수	0 1 1 1
NAND	음수	큰 양수	1 1 1 0
NOR	음수	NAND보다 작은 양수	1 0 0 0
XOR	단층으로 불가능 (학습하면 가중치·편향이 0 근처)		0 1 1 0

활성화 함수

- 다음 뉴런으로 신호를 얼마나 전달할지 결정. **비선형성**이 있어야 층을 쌓는 의미가 있음 (선형 함수만 쌓으면 결국 하나의 선형 함수).
- **Step**: x ≥ 0이면 1, 아니면 0. 미분값이 0이라 경사하강법 학습에는 부적합.
- **Sigmoid** $1/(1+e^{-x})$: 0~1 연속값 → 확률로 해석 가능 → 이진 분류 출력층.
- **ReLU** max(0, x): 계산이 빠르고 CNN 은닉층의 기본. 음수(원치 않는 특징)는 0, 양수는 그대로.

11. 학습 원리

- **학습**: 예측과 정답의 오차, 즉 **손실(Loss)**을 줄이는 방향으로 가중치·편향을 반복 수정.
- **손실 함수**: 예측과 정답의 차이 (작을수록 좋음). 이진 분류는 Sigmoid + **Binary Cross-Entropy**, 다중 분류는 **Cross Entropy** (logit을 그대로 입력, Softmax는 내부에 포함).
- **기울기 (Gradient)**: 가중치를 바꿨을 때 손실이 어떻게 변하는지. **연쇄법칙** dL/dw = dL/dy · dy/dz · dz/dw
- **순전파**: 입력 → 출력, 예측 / **역전파**: 출력 → 입력으로 거슬러 올라가며 각 층의 기울기 계산 후 가중치 수정.
- **경사하강법**: $w_{new} = w_{old} - \eta \cdot dL/dw$ — 기울기의 반대 방향으로 이동해서 손실 감소.
- **Optimizer** (SGD, Adam 등): gradient로 실제 파라미터를 수정. 배치마다 `zero_grad`로 초기화 (PyTorch는 gradient를 누적하기 때문).
- **Batch / DataLoader**: 데이터를 묶음(Batch)으로 입력. DataLoader는 배치 생성, 반복, 셔플, 로딩 관리를 맡음.
- **평가 단계**: 가중치를 수정하지 않음. `eval()`과 `no_grad`로 gradient 계산을 끔 → 메모리·속도 절약.

	너무 작거나 적으면	너무 크거나 많으면
학습률 η	학습이 매우 느림, 최적값 도달에 오래 걸림	최적값을 지나치거나 발산, 불안정
Epoch	**과소적합** (충분히 학습하지 못함)	**과적합** (학습 데이터만 외우고 새 데이터에 일반화 실패)

12. CNN (합성곱 신경망)

- 이미지를 평평하게 펴지 않고 2D/3D 구조를 유지한 채 주변 픽셀 사이의 공간 패턴을 추출. 같은 커널을 이미지 전체에 공유 → 파라미터가 적음.
- **Convolution**: 커널로 특징 추출 → **Feature Map** (커널 1개당 1장). CNN의 **가중치 = 커널 각 칸의 값**, **편향** = 약한 특징을 무시하거나 증폭하는 값.
- 전통적 필터링과의 차이: Sobel 같은 필터는 사람이 커널을 설계하고, CNN은 커널을 **학습**.
- **Feature Map 값**: 큰 양수 = 패턴이 강함 / 0 근처 = 특징 없음 / 큰 음수 = 반대 방향 패턴.
- **ReLU** → 음수 제거 / **Pooling** → 중요한 특징은 유지하면서 크기 축소 (Max Pool 2×2 → 가로세로 절반), 연산 효율 향상, 위치 변화에 강해짐 / **Flatten + FC** → 최종 분류.
- 얕은 층은 선·경계·색, 깊은 층은 질감·부분 형태·객체 구조를 잡음 → 깊어질수록 채널 수 증가 (64 → 128 → 256).
- 학습 전 필터는 무작위라서 Feature Map에 의미가 없고, 학습 후에는 획이나 방향에 반응.

출력 크기 공식 $\lfloor (H + 2P - K) / S \rfloor + 1$ (H 입력 크기, P padding, K 커널, S stride)
예: 28×28, K=3, P=1, S=1 → 28×28 | Max Pool 2×2 (S=2) → 14×14

MNIST SimpleCNN (입력 1×28×28, 학습 60,000장 / 테스트 10,000장)
1×28×28 → Conv1 8×28×28 → ReLU → Pool 8×14×14 → Conv2 16×14×14 → ReLU → Pool 16×7×7 → Flatten 784 → FC 64 → FC 10
파라미터 수 **52,138** = conv1 (1·8·9 + 8 = 80) + conv2 (8·16·9 + 16 = 1,168) + fc1 (784·64 + 64 = 50,240) + fc2 (64·10 + 10 = 650)

CIFAR-10에서 추가된 기법 (3×32×32 RGB, 10 클래스, 학습 50,000장 / 테스트 10,000장)

기법	역할	적용 대상
데이터 증강 (RandomCrop, Flip)	위치·방향을 무작위로 변형 → 데이터 다양성 증가, 과적합 완화	**학습 데이터에만**
정규화 (채널별 mean·std)	입력 분포를 맞춰 gradient와 optimizer를 안정화. 정규화 후 값 범위가 약 −2~2라서 시각화할 때 역정규화 필요	**학습·테스트 모두** (같은 입력 분포를 보도록)
BatchNorm2d	각 층의 출력(Feature Map) 분포를 정규화 → 학습 안정화	Conv 뒤
AdaptiveAvgPool2d	Feature Map을 평균값 하나로 압축 (256×4×4 → 256×1×1) → FC 파라미터 감소	FC 앞
Dropout	학습 중 일부 뉴런을 무작위로 끔 → 특정 특징에 의존하지 않게 해서 과적합 완화	FC 부근

13. GPU 병렬 연산

	CPU (NumPy)	GPU (CuPy / PyTorch CUDA)
코어	소수의 고성능 코어 (Orin Nano 6개)	다수의 코어 (1024개)
처리	복잡한 제어 흐름을 순차 처리	단순한 연산을 대규모 병렬 처리
메모리	RAM (host)	VRAM (device)

- Jetson은 **UMA (통합 메모리)**로 물리 칩을 공유하지만 논리 영역은 분리 → 서로 다른 장치의 데이터를 함께 연산하려면 **복사가 필요** (NumPy 배열과 CuPy 배열을 섞으면 에러). 같은 이유로 PyTorch에서도 **모델과 텐서는 같은 device**에 있어야 함.

- **GPU가 느릴 수 있는 경우**: 작은 데이터에서는 전송·실행 오버헤드가 연산 이득보다 큼. 첫 실행에는 초기화 비용이 들어서 **warm-up**이 필요. GPU는 비동기로 실행되므로 시간을 정확히 재려면 **synchronize**로 완료를 기다려야 함.
- **병렬화 방식** (잔상 효과, convolution): CPU는 출력 픽셀마다 반복 (H×W번). GPU는 커널 원소(이동량)만큼만 반복하고, 각 반복에서 배열 전체를 한 번에 곱하고 누적. 계산 순서는 달라도 결과는 동일.
- **CuPy**: NumPy 호환 GPU 배열 라이브러리, CUDA로 가속.
- **PyTorch**: GPU를 지원하는 딥러닝 프레임워크. Tensor 사용, **자동 미분 (autograd)**, nn / optim / DataLoader 제공. `ToTensor`는 (H,W,C) → (C,H,W)로 바꾸고 0~255 → 0~1 float32로 변환.

시험에 잘 나올 비교 쌍

LPF vs HPF · Sobel vs Canny · Erosion vs Dilation · Opening vs Closing · HE → AHE → CLAHE · HSV vs LAB · RGB vs BGR · SLP vs MLP · Step / Sigmoid / ReLU · 과소적합 vs 과적합 · 학습률이 클 때 vs 작을 때 · 순전파 vs 역전파 · 데이터 증강 vs 정규화 · CPU vs GPU · USB vs CSI 카메라 · Kalman 예측 vs 보정

강의 자료에서 틀렸거나 헷갈리는 부분
① DL 노트북 A-2의 계단 함수 수식은 0과 1이 뒤바뀐 오타 → 올바른 정의는 **$z \geq 0$일 때 1** ② CV 노트북 Sobel 설명에서 dy 항목에 "dx=1 설정"이라고 적힌 것은 오타 → **dy=1** ③ H 순환 코드 `h += 100 % 360`은 100만 더할 뿐 순환이 안 됨 → **(h + 100) % 360** ④ CLAHE 주석에는 clip 2.0, 코드는 3.0 → 시험에서는 문제에 주어진 값을 사용